

1 Creating a New Column in a Dataset

Suppose we have many **customer reviews** in our dataset.

A review can be:

- Very short: "Great!"
- Very long: several sentences or even paragraphs

Sometimes it is important to know **how long a review is (how many words it contains)** because:

- Very short reviews may not contain enough information.
- Very long reviews may need to be handled differently.

So, we will calculate the **word count of each review**.

2 Function to Calculate Word Count

First, we write a simple function.

```
def compute_review_length(example):  
    return {"review_length": len(example["review"].split())}
```

Let's understand what is happening here.

example

`example` represents **one row of the dataset**.

For example, a row might look like this:

```
{  
  'review': "It has no side effect I take it daily"  
}
```

`example["review"]`

This extracts the **review text**.

"It has no side effect I take it daily"

split()

```
example["review"].split()
```

This **splits the sentence into words using spaces**.

Result:

```
["It", "has", "no", "side", "effect", "I", "take", "it", "daily"]
```

len()

```
len(...)
```

This counts the **number of words**.

```
9
```

return

Finally, the function returns a dictionary:

```
{"review_length": 9}
```

This means a **new column called review_length will be created**.

3 Adding the New Column Using Dataset.map()

Now we apply this function to the dataset.

```
drug_dataset = drug_dataset.map(compute_review_length)
```

What does **map()** do?

`Dataset.map()` applies a function to every row in the dataset.

As a result, a new column is added to each row.

Example Result

```
drug_dataset["train"][0]
```

Output:

```
{
  'patient_id': 206461,
  'drugName': 'Valsartan',
  'condition': 'left ventricular dysfunction',
  'review': '"It has no side effect, I take it in combination of Bystolic 5 Mg and Fish Oil"',
  'rating': 9.0,
  'date': 'May 20, 2012',
  'usefulCount': 27,
  'review_length': 17
}
```

Here we can see the new column:

```
review_length: 17
```

This means the review contains **17 words**.

4 Sorting Reviews Using Dataset.sort()

Now we want to check:

- Which reviews are the **shortest**
- Which reviews are the **longest**

So we sort the dataset.

```
drug_dataset["train"].sort("review_length")[:3]
```

Here:

```
sort("review_length")
```

means **sort the dataset based on review_length**.

Then:

```
[ :3]
```

means **show the first 3 rows**.

Output Example

```
review: "Excellent."  
review_length: 1
```

```
review: "useless"  
review_length: 1
```

```
review: "ok"  
review_length: 1
```

These reviews contain **only one word**.

The Problem

These reviews might be acceptable for **sentiment analysis**.

But if we want to **predict the medical condition**, they are not very informative.

5 Alternative Method: add_column()

Another way to add a column is using:

```
Dataset.add_column()
```

This method is useful when:

- You already have the values in a **Python list**
- Using `map()` is not convenient

Example:

```
dataset.add_column("review_length", list_of_lengths)
```

6 Filtering Out Very Short Reviews

Now we want to **remove very short reviews**.

For example:

Remove reviews with fewer than 30 words

Code

```
drug_dataset = drug_dataset.filter(lambda x: x["review_length"] > 30)
```

What does **filter()** do?

It **keeps only the rows that satisfy a condition**.

The condition here is:

```
review_length > 30
```

So:

✓ 31 words → kept

✓ 50 words → kept

✗ 10 words → removed

Dataset Size After Filtering

```
print(drug_dataset.num_rows)
```

Output:

train: 138514

test: 46108

This means about **15% of the reviews were removed**.

7 Finding the Longest Reviews

If we want to see the **largest reviews**, we sort in descending order.

```
drug_dataset["train"].sort("review_length", reverse=True)
```

Here:

```
reverse=True
```

means **descending order**.

So the order becomes:

largest → smallest

8 HTML Character Problem

Some datasets contain **HTML character codes**.

For example:

```
I&#039;m happy
```

Here:

```
&#039;
```

represents the character:

,

Example

```
import html
```

```
text = "I&#039;m a transformer called BERT"  
html.unescape(text)
```

Output:

I'm a transformer called BERT

9 Cleaning All Reviews in the Dataset

Now we clean all reviews in the dataset.

```
drug_dataset = drug_dataset.map(  
    lambda x: {"review": html.unescape(x["review"])}  
)
```

Here:

- 1 Each review is taken from the dataset
- 2 `html.unescape()` removes HTML codes
- 3 The cleaned review is saved back into the dataset